Solving N-Queens with SAT: An Experimental Comparison of Cardinality Encodings and Exact Solvers

Do Duc Long[a]

[a]VNU University of Engineering and Technology, Hanoi, Vietnam

ARTICLE INFO

Keywords:
N-queens
SAT encoding
cardinality constraints
constraint programming
integer linear programming

ABSTRACT

This report implements an exact SAT solver for the standard N-queens puzzle and compares five at-most-one encodings: pairwise, binary, commander, recursive product, and sequential counter. A minimal formulation and a formulation with redundant column clauses are evaluated using PySAT. Four SAT backends are compared using a fixed encoding, followed by comparisons with IBM CPLEX CP Optimizer, IBM CPLEX MIP, Gurobi, and OR-Tools CP-SAT. The implementation uses the same problem semantics for every method, independently validates returned placements, and stores the raw solutions and experiment configurations. Experiments use fresh processes, one solver thread, three repetitions, and an enforced ten-second process deadline. Boards up to 512×512 are included to examine scaling. Results distinguish model construction and search from process startup and license acquisition, and account explicitly for timeouts. Commander/minimal achieves the lowest encoding PAR-2 (0.450 s). At $n = 512$, CPLEX CP and both MIP baselines succeed in all repetitions, while reference SAT and CP-SAT reach the deadline. The study concerns finding one solution to the unconstrained puzzle; it does not measure counting or the NP-complete completion problem.

1. Introduction

The N-queens puzzle asks for n queens on an $n \times n$ chessboard such that no two queens attack each other. It provides a compact example of exact constraint solving: every row must contain a queen, while columns and diagonals must exclude conflicts. A Boolean placement model turns these requirements into cardinality constraints. Its runtime can depend on how those constraints are translated into conjunctive normal form (CNF), even when the resulting formulas have exactly the same board solutions.

PySAT supplies a common interface to SAT engines and cardinality encoders [6, 8]. This makes it possible to compare encoding choices without changing the application semantics. Constraint programming (CP) instead represents a queen by its column and expresses non-attacking conditions with global all-different constraints [3]. Integer linear programming (ILP) represents occupied squares with binary variables and linear equalities or inequalities. These alternative formulations allow a comparison between SAT and established exact solvers.

The present work contributes (i) an executable SAT solver with five AMO encodings and two formulations, (ii) a semantic verification suite that checks complete projected solution sets on small boards, (iii) comparisons with four SAT backends and all four CP/ILP solvers required by Homework 1, and (iv) reproducible measurements, raw placements, figures, and this report.

The standard puzzle has a solution for $n = 1$ and every $n \geq 4$, while $n = 2$ and $n = 3$ are infeasible. Consequently, deciding existence in this particular family should not be called NP-complete. The N-queens completion problem, where some queens are fixed in advance, is a different problem and is NP-complete [2]. Here the aim is to study the behavior of generic exact methods on the standard puzzle, rather than propose a new algorithm that supersedes explicit constructions.

Email address: 24021550@vnu.edu.vn (D.D. Long)

2. Problem definition and baseline models

2.1. Problem definition

Let $I = \{0, \dots, n-1\}$. A placement is a vector $q = (q_0, \dots, q_{n-1})$ where $q_r \in I$ is the column of the queen in row r. It is feasible iff

$$q_r \neq q_s \qquad \forall r < s, \tag{1}$$

$$q_r - r \neq q_s - s \qquad \forall r < s, \tag{2}$$

$$q_r + r \neq q_s + s \qquad \forall r < s. \tag{3}$$

There is no optimization objective: the first feasible placement suffices. All models below describe precisely these placements. Figure 1 shows a placement returned during the experiments. Rows and columns in the figure are labeled from one; source code and saved solutions use zero-based coordinates.

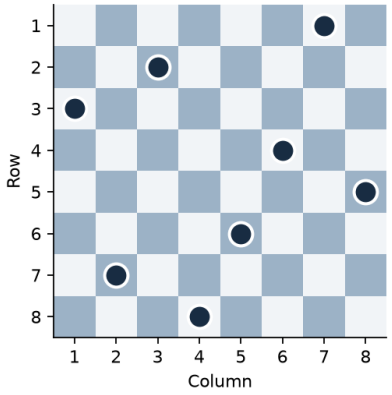


Figure 1: An independently validated solution of the 8×8 puzzle. Each dot denotes one queen; no two dots share a column or diagonal.

2.2. Constraint programming

The CP formulation introduces n integer variables $q_r \in I$ and the three global constraints

$$\text{AllDifferent}(q_0, \dots, q_{n-1}), \tag{4}$$

$$\text{AllDifferent}(q_0, q_1 + 1, \dots, q_{n-1} + n - 1), \tag{5}$$

$$\text{AllDifferent}(q_0, q_1 - 1, \dots, q_{n-1} - (n-1)). \tag{6}$$

The same expressions are supplied to CPLEX CP Optimizer through DOcplex [5] and to OR-Tools CP-SAT. The counts n variables and three constraints refer to the user model; solver-internal expansion and presolve may add variables and constraints. In particular, CP-SAT is not a pure finite-domain CP engine: it combines SAT-based reasoning with other techniques. Its inclusion provides an exact baseline through a different modeling interface, rather than a disjoint category of algorithms.

2.3. Binary ILP formulation

Define $x_{rc} \in \{0, 1\}$ for each $(r, c) \in I^2$. Let $D_d^- = \{(r, c) : r - c = d\}$ and $D_s^+ = \{(r, c) : r + c = s\}$. The feasibility model is

$$\sum_{c \in I} x_{rc} = 1 \qquad \forall r \in I, \tag{7}$$

$$\sum_{r \in I} x_{rc} = 1 \qquad \forall c \in I, \tag{8}$$

$$\sum_{(r,c)\in D_d^-} x_{rc} \le 1 \qquad \forall d \,:\, |D_d^-| \ge 2, \tag{9}$$

$$\sum_{(r,c)\in D_s^+} x_{rc} \le 1 \qquad \forall s \,:\, |D_s^+| \ge 2. \tag{10}$$

The constant objective is zero. For $n \ge 2$ there are n^2 binary variables and $6n-6$ linear constraints. Singleton diagonals need no constraint. Gurobi [4] and CPLEX MIP receive the identical model, with one thread, an explicit seed, and the default presolve settings. No model receives symmetry-breaking constraints, a warm start, or a precomputed placement.

3. SAT encoding

3.1. Board variables and formulations

The Boolean variable x_{rc} means that square (r, c) contains a queen and has DIMACS index $rn + c + 1$. For a literal set X, define

$$\mathrm{ALO}(X) = \bigvee_{x\in X} x, \qquad \mathrm{EO}(X) = \mathrm{ALO}(X) \wedge \mathrm{AMO}(X). \tag{11}$$

Write R_r, C_c, and $\mathcal{D}$ for row, column, and nonsingleton diagonal literal sets. The minimal formula is

$$F_{\min} = \bigwedge_r \mathrm{EO}(R_r) \;\wedge\; \bigwedge_c \mathrm{AMO}(C_c) \;\wedge\; \bigwedge_{D\in\mathcal{D}} \mathrm{AMO}(D). \tag{12}$$

The extended formula adds column at-least-one clauses:

$$F_{\mathrm{ext}} = F_{\min} \wedge \bigwedge_c \mathrm{ALO}(C_c). \tag{13}$$

Both formulas have exactly the same feasible board placements. Every model of $F_{\min}$ has exactly n queens by row EO. Column AMO distributes these queens among n columns with no repetition, so every column is occupied. Diagonal AMO then gives Equations (2)–(3). Conversely, every valid placement satisfies all constraints in both formulations, and the auxiliary variables of each correct AMO encoding can be extended to satisfy its CNF. The extended formula contains n additional clauses and no additional variables. Although logically redundant, those clauses may change propagation and search.

3.2. Five at-most-one encodings

For $X = \{x_1, \dots, x_k\}$, the implementations are as follows.

Pairwise. For every $i < j$, add $\neg x_i \vee \neg x_j$. This needs $\binom{k}{2}$ clauses and no auxiliary variables. It is implemented directly by enumerating pairs.

Binary. Introduce $\lceil \log_2 k \rceil$ bits and assign a distinct bit pattern to every literal. Each x_i implies its pattern through binary clauses. Two true literals would force different values for some bit. The implementation uses PySAT's bitwise cardinality encoder [8].

Commander. Partition literals into groups of at most three. For a group G introduce a_G, add pairwise AMO within G, add $\neg x \vee a_G$ for $x \in G$, and add $\neg a_G \vee \bigvee_{x\in G} x$. Apply the construction recursively to the commanders [7]. Groups with at most four literals are handled by pairwise encoding. The reverse implication prevents an unused commander from being true; it is included consistently in all experiments.

Recursive product. Arrange the k literals in a $p \times q$ grid, with $p = \lceil \sqrt{k} \rceil$ and $q = \lceil k/p \rceil$. Introduce row indicators u_i and column indicators v_j. A literal in position (i, j) implies both u_i and v_j. Encode AMO on the two indicator sets recursively, using pairwise constraints for sets of size at most four [1]. Distinct true literals must differ in a row or a column, contradicting one indicator AMO.

Table 1
Asymptotic size of one AMO constraint. Small-group shortcuts in the actual implementations can change exact counts.

Encoding	Auxiliary variables	Clauses
Pairwise	0	$\binom{k}{2}$
Binary	$\lceil \log_2 k \rceil$	$k\lceil \log_2 k \rceil$
Commander	$O(k)$	$O(k)$
Recursive product	$O(\sqrt{k})$	$O(k)$
Sequential counter	$k-1$	$3k-4$ $(k \geq 3)$

Sequential counter. PySAT supplies Sinz's sequential counter [9]. For $k \geq 3$, it uses $k-1$ auxiliary variables and $3k-4$ clauses for AMO. The clauses propagate prefix occupancy and exclude any later true literal once the prefix is occupied. PySAT simplifies the two-literal case to one binary clause.

There are $O(n)$ AMO groups with up to n literals each. The complete pairwise CNF therefore has $\Theta(n^3)$ clauses; binary uses $O(n^2 \log n)$, while commander, product, and sequential counter use $O(n^2)$. More precisely, pairwise $F_{\min}$ has

$$n + 2n\binom{n}{2} + 2\left(\binom{n}{2} + 2\binom{n}{3}\right) = \frac{5n^3 - 6n^2 + 4n}{3} \tag{14}$$

clauses for $n \geq 2$. A compact formula is not sufficient to predict runtime: propagation and the resulting search trajectory also matter.

3.3. Implementation and correctness checks

The SAT encoder streams clauses into the chosen solver instead of retaining a second full Python copy of the formula. An export command produces standard DIMACS CNF for independent use. Decoding inspects only the first n^2 placement variables, so auxiliary assignments cannot create queens. Every SAT or CP/ILP solution passes a separate checker for length, domain, column uniqueness, and uniqueness of both diagonal indices.

The semantic test suite enumerates every truth assignment for AMO groups of size zero through eleven, for both positive and negative literals. It then enumerates all projected N-queens solutions for $n = 1, \ldots, 8$ for all ten SAT configurations. Blocking clauses use only selected board squares, so distinct auxiliary assignments cannot be counted as distinct placements. The resulting solution sets are compared with an independent backtracking implementation, yielding counts $1, 0, 0, 2, 10, 4, 40, 92$. Integration checks also confirm infeasibility at $n = 3$ and a valid solution at $n = 8$ for every CP/ILP backend. These tests verify semantic equivalence rather than only checking that the code runs.

4. Computational experiments

4.1. Environment and protocol

All runs were performed on Windows 11 (build 26200), with an AMD Ryzen 7 8845HS processor (16 logical processors), Python 3.14.5, PySAT 1.9.dev2, Gurobi 13.0.3, CPLEX Python package 22.2.0.1, DOcplex 2.32.264, and OR-Tools 9.15.6755. CPLEX CP uses the CP Optimizer executable from Optimization Studio 22.2. The saved configuration files record these versions and the complete command parameters. Gurobi uses a working WLS license; CPLEX MIP was verified to accept models beyond the community-edition size limit.

Each experiment runs sequentially in fresh child processes, with one solver thread and three repetitions. CP/ILP runs use seeds one through three. SAT backends retain their default parameters; their repetitions primarily measure timing variation and are not different randomized SAT seeds. Method order is rotated across repetitions. Every run has a hard ten-second deadline for the child process, including imports, license acquisition, construction, and solving. Exceeding this budget yields TO; it never implies infeasibility. The Windows harness also terminates child CP Optimizer processes on timeout. Recorded timeout duration includes termination and cleanup; operating-system delays can make this exceed the requested budget. Such records remain unfinished runs, regardless of the cleanup duration.

Two timing measures are kept: process time, measured by the parent, and core time, equal to model construction plus solver execution. Gurobi WLS startup is recorded separately as setup time; it is included in process time but excluded from core time. The main comparison uses process time to reflect the budget actually enforced. Both timings

are reported to avoid attributing all license and import overhead to search performance. Construction and solve times cannot be recovered when the process is killed before returning; these fields remain empty rather than fabricated.

For successful configurations, per-size tables show the median of three process times. A cell TO ($s/3$) means only s of three runs finished. Aggregate rankings use PAR-2: a successful run contributes its process time, and an unfinished or erroneous run contributes twice the ten-second budget. This prevents fast successful subsets from hiding failures. The benchmark contains one standard board per size; repetitions are repeated measurements, not distinct instances or independent evidence of broad statistical significance.

4.2. Encoding and formulation comparison

The first experiment fixes CaDiCaL 1.9.5 and evaluates all five AMO encodings under both formulations at $n \in \{8, 16, 32, 64, 96, 128\}$. It contains $6 \times 10 \times 3 = 180$ runs. Table 2 includes exact variable and clause counts at $n = 128$, recomputed from the encoder independently of whether a particular timed run finished. Figure 2 shows PAR-2 by board size.

Table 2
Encoding comparison over 18 runs per configuration. Structural counts refer to the 128×128 board. Lower PAR-2 is better.

Encoding / formulation	Solved	PAR-2 (s)	Variables	Clauses
commander/minimal	18/18	0.450	49,354	226,242
commander/extended	18/18	0.459	49,354	226,370
product/extended	18/18	0.921	39,174	169,112
binary/extended	18/18	1.221	21,234	447,828
seqcounter/extended	18/18	1.275	81,150	193,800
pairwise/minimal	18/18	1.863	16,384	3,462,656
pairwise/extended	18/18	1.898	16,384	3,462,784
binary/minimal	18/18	3.113	21,234	447,700
seqcounter/minimal	15/18	5.718	81,150	193,672
product/minimal	12/18	7.534	39,174	168,984

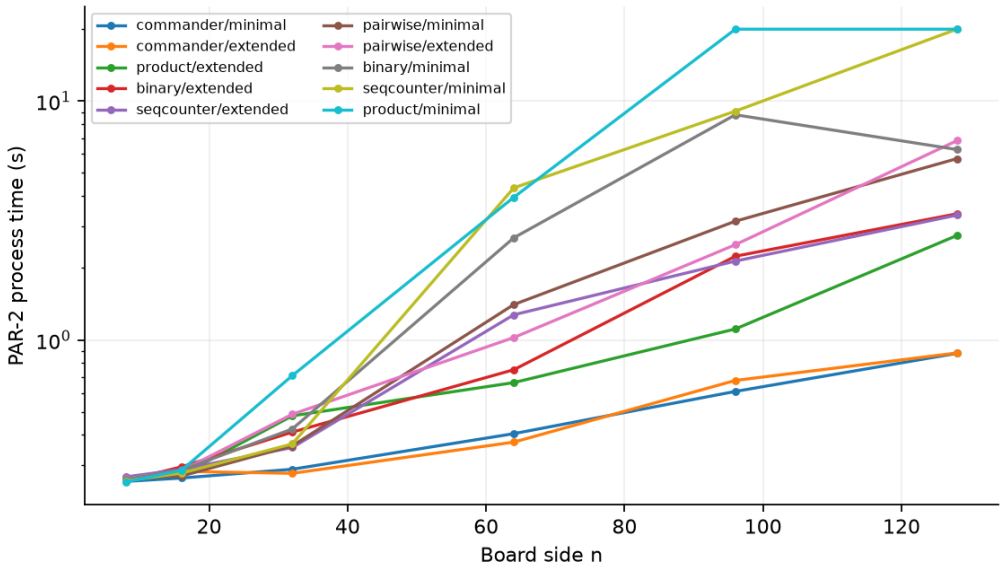


Figure 2: Encoding and formulation comparison with CaDiCaL. Timeouts contribute 20 seconds each to PAR-2.

The lowest aggregate PAR-2 is obtained by commander/minimal, at 0.450 seconds, with 18 of 18 runs solved. At $n = 128$, pairwise/minimal has 3,462,656 clauses, whereas commander/minimal has 226,242. The ranking is measured

rather than inferred from clause count alone. The reference used in the following experiments is commander/minimal; it was selected after an exploratory encoding comparison on the same size family. The reported encoding results were then collected with the finalized harness. For product, the minimal formulation solves 12/18 runs with PAR-2 7.534 seconds; adding column ALO solves 18/18 with PAR-2 0.921 seconds. For seqcounter, the minimal formulation solves 15/18 runs with PAR-2 5.718 seconds; adding column ALO solves 18/18 with PAR-2 1.275 seconds. Thus the added column clauses can improve measured runtime without changing the set of placements, although the effect depends on the encoding.

4.3. SAT solver comparison

The second experiment fixes the selected reference encoding and compares CaDiCaL 1.9.5, Glucose 4.2, MiniSat 2.2, and MapleChrono through PySAT. These backends were checked locally before benchmarking. The same six sizes and three repetitions give $6 \times 4 \times 3 = 72$ runs. The selected encoding is disclosed below; choosing it on the same board family is an in-sample engineering choice rather than a held-out prediction.

Table 3
Median process time (seconds) for the four SAT backends with a fixed reference encoding. TO includes the number of completed runs.

n	cadical195	glucose42	minisat22	maplechrono
8	0.253	0.256	0.260	0.255
16	0.257	0.293	0.273	5.354
32	0.278	7.891	0.286	6.866
64	0.359	4.073	0.364	TO (0/3)
96	0.637	TO (0/3)	0.513	TO (0/3)
128	0.846	TO (0/3)	1.956	TO (0/3)

All backends use commander/minimal. Their aggregate PAR-2 values are cadical195: 0.441 seconds, minisat22: 0.609 seconds, glucose42: 8.755 seconds, maplechrono: 12.081 seconds. These results describe this fixed formula family and short budget; they do not establish a universal ranking of SAT engines.

4.4. SAT versus CP and ILP

The third experiment compares the reference SAT configuration with CPLEX CP, CPLEX MIP, Gurobi, and CP-SAT. It uses the six original sizes plus $n = 256$ and $n = 512$, retaining the same three repetitions, thread count, and ten-second process deadline. There are $8 \times 5 \times 3 = 120$ runs in the combined main and scaling experiments.

Table 4
Median process time (seconds) for exact methods. Every fully solved cell represents three validated placements.

n	SAT	CPLEX CP	CPLEX MIP	Gurobi	CP-SAT
8	0.247	1.510	0.401	0.345	1.325
16	0.264	1.496	0.427	0.357	1.354
32	0.272	1.443	0.427	0.344	1.370
64	0.363	1.466	0.467	0.423	1.725
96	0.685	1.462	0.517	0.412	2.529
128	0.871	1.503	0.606	0.700	3.838
256	4.085	1.419	1.248	0.854	TO (0/3)
512	TO (0/3)	1.440	5.617	2.367	TO (0/3)

Table 5
Median construction-plus-solve time (seconds) for the same runs. Imports and Gurobi WLS acquisition are excluded; deadline status is unchanged.

n	SAT	CPLEX CP	CPLEX MIP	Gurobi	CP-SAT
8	0.002	0.040	0.005	0.004	0.011
16	0.005	0.039	0.009	0.005	0.027
32	0.024	0.039	0.017	0.010	0.089
64	0.101	0.041	0.053	0.079	0.443
96	0.364	0.046	0.108	0.056	1.223
128	0.569	0.049	0.194	0.338	2.538
256	3.610	0.061	0.848	0.403	TO (0/3)
512	TO (0/3)	0.092	5.090	1.530	TO (0/3)

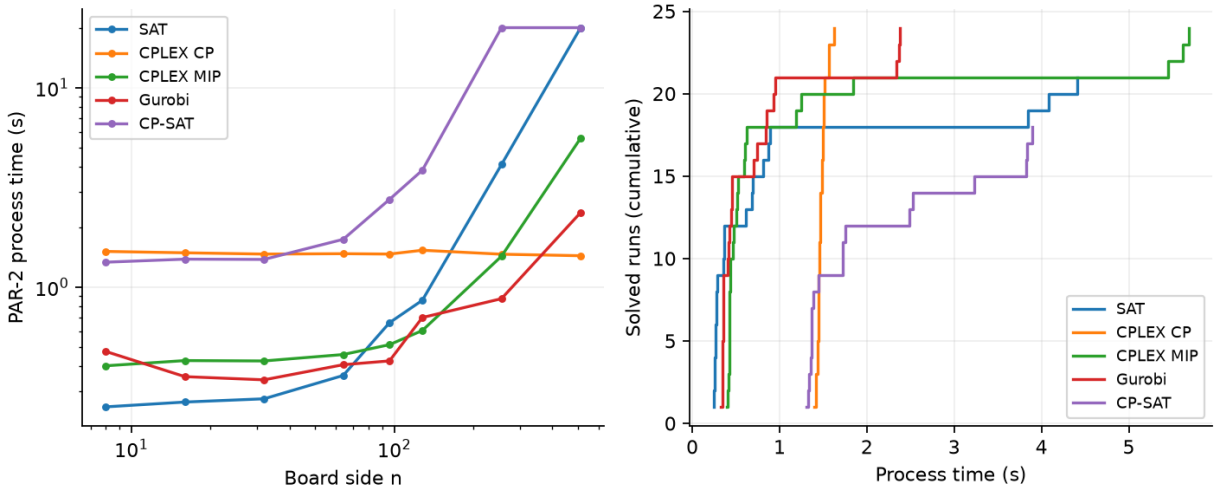


Figure 3: Exact-method comparison: PAR-2 versus board size (left) and cumulative solved runs versus process time (right). The cumulative plot counts repetitions rather than distinct sizes.

Across the 24 runs per method, SAT solves 21/24 with PAR-2 3.348 seconds; CPLEX CP solves 24/24 with PAR-2 1.477 seconds; CPLEX MIP solves 24/24 with PAR-2 1.230 seconds; Gurobi solves 24/24 with PAR-2 0.743 seconds; CP-SAT solves 18/24 with PAR-2 6.553 seconds. Timeout cells in Tables 4 and 5 preserve unsuccessful runs. On small boards, process times include library and license startup that may exceed actual search time. On larger boards, representation size and search dominate, so the compact CP model and binary board model need not scale similarly. At $n = 512$, SAT completes 0/3 runs; CPLEX CP completes 3/3 runs; CPLEX MIP completes 3/3 runs; Gurobi completes 3/3 runs; CP-SAT completes 0/3 runs. The CP user model at this size has 512 integer variables and three global constraints; the ILP user model has 262,144 binary variables and 3,066 linear constraints. These are input-model counts, not the numbers remaining after presolve.

4.5. Limitations

The ten-second deadline measures short-budget behavior, so a TO is not evidence that a method cannot solve that size with more time. Core times should not be read as successful runs under a separate ten-second search budget: all outcomes were determined by the same process deadline. Startup overhead is material on small boards, especially for CP-SAT imports and Gurobi WLS acquisition. Hardware power management and background tasks may introduce timing noise despite sequential execution.

Only unconstrained square boards and default solver settings are studied. The selected SAT encoding is optimized using the same family later used for comparison, and the report does not claim an unbiased general ranking of solver technology. In particular, better CP, ILP, or SAT formulations, specialized constructions, symmetry breaking, or

different parameter settings could change the ordering. Large-board tests concern finding one placement, not exhaustive counting. Completion instances would require additional benchmarks and conclusions.

5. Conclusion

The delivered implementation solves standard N-queens through SAT and four required CP/ILP baselines, exports CNF, independently checks placements, and reproduces the measured comparisons. The encoding study separates formula size from observed runtime, while the formulation study isolates the effect of redundant column clauses. Process and core timings make startup costs explicit. Complete small-board solution-set tests support the correctness of all ten SAT configurations, and saved placements permit independent validation of every successful benchmark run. The final audit covers 372 timed runs and independently revalidates 339 returned placements. There are 33 timeouts and 0 solver errors. The measured differences support selecting a suitable encoding and modeling interface for the available budget; they do not imply that any generic solver replaces specialized N-queens constructions.

Data availability

Source code, tests, raw CSV results, JSONL placements, configuration files, and report sources are available at `https://github.com/dolong226/homework1-nqueens`. Gurobi license credentials are local configuration and are excluded from the repository.

Declaration of generative AI and AI-assisted technologies

OpenAI Codex assisted with implementation, experiment automation, and report drafting. The automated correctness and solution-validation checks are documented above. The student should review the code, results, and manuscript before submission.

References

[1] Chen, J., 2010. A new SAT encoding of the at-most-one constraint. Workshop on Constraint Modelling and Reformulation (ModRef).

[2] Gent, I.P., Jefferson, C., Nightingale, P., 2017. Complexity of n-queens completion. Journal of Artificial Intelligence Research 59, 815–848. doi:10.1613/jair.5512.

[3] Google, n.d. The N-queens problem: OR-Tools. https://developers.google.com/optimization/cp/queens. Accessed 10 October 2026.

[4] Gurobi Optimization, n.d. Gurobi optimizer reference manual. https://docs.gurobi.com/projects/optimizer/en/current/. Version 13.0.3 used in the experiments.

[5] IBM, n.d. DOcplex constraint programming model documentation. https://ibmdecisionoptimization.github.io/docplex-doc/cp/docplex.cp.model.py.html. Accessed 10 October 2026.

[6] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, pp. 428–437. doi:10.1007/978-3-319-94144-8_26.

[7] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects. Workshop on Constraints in Formal Verification.

[8] PySAT Developers, n.d. Cardinality encodings: pysat.card. https://pysathq.github.io/docs/html/api/card.html. Accessed 10 October 2026.

[9] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, pp. 827–831. doi:10.1007/11564751_73.